

Cave & Dungeon Toolkit Manual

[Importing the package:](#)

[Using the assets](#)

[Cave and Ice Prefabs:](#)

[Using the Tools](#)

[Vertex Influence Spheres](#)

[Advanced Vertex Influence Sphere Info:](#)

Visit WaltWW.com for more information and support.

Installation Notes

To see the vertex influence sphere gizmos, move the “Gizmos” folder to the top level directory of your project, so that it resides in “Assets/Gizmos”.

No matter if you’re using HDRP, URP or the Built-In Render Pipeline, there are no additional steps you need to take besides extracting the package to start creating your own scenes with these assets. To use the included demo scenes, however, there are some additional steps needed depending on the render pipeline.

HDRP

The example scenes were designed for HDRP and should work out of the box for a properly configured HDRP project. The one issue you may experience is the scene being very bright. Simply press the play button once and this should be fixed.

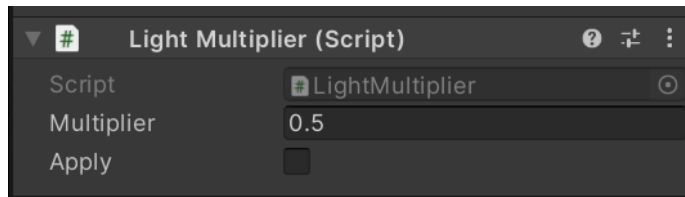
URP and Built-In Render Pipeline

The example scenes were **not** originally designed for these render pipelines, so please be aware you will need to do some additional work to get these working properly in these render pipelines.

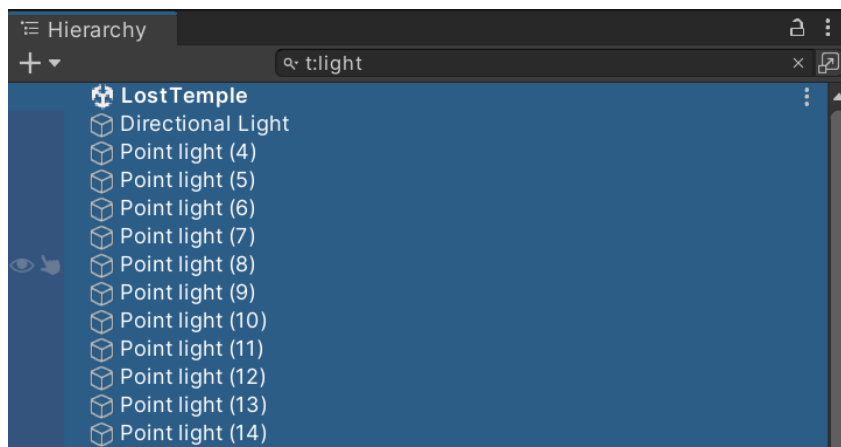
The steps needed for both URP and the Built-In Render Pipeline are basically the same, so they will be listed out with notes about any differences exclusive to one pipeline:

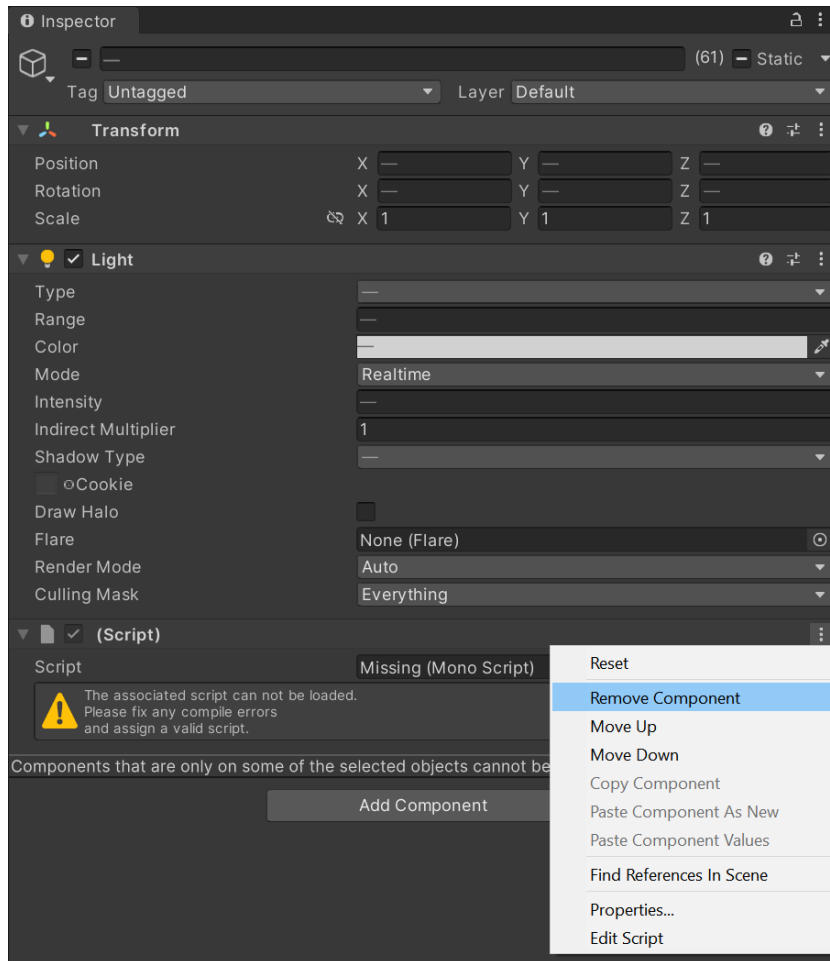
If all shaders are pink, make sure ShaderGraph is installed via the package manager. Note you need Unity version 2021.2 or later for ShaderGraph to work with the Built-In Render Pipeline.

Scenes might be too bright. Use the included LightMultiplier.cs script to dim all the lights in the scene. Create a new gameobject, select Add Component and type "Light Multiplier". Once the script has been added, hit apply a few times and you should see the lights dim. Change the multiplier for more fine tuned adjusting, and this script allows you to undo as well.



Lights and cameras will be missing scripts. These are the AdditionProperties scripts that HDRP adds. You can delete these. (Tip: type "t:light" and "t:camera" into the hierarchy search bar, select all and delete the missing script to speed this up.)





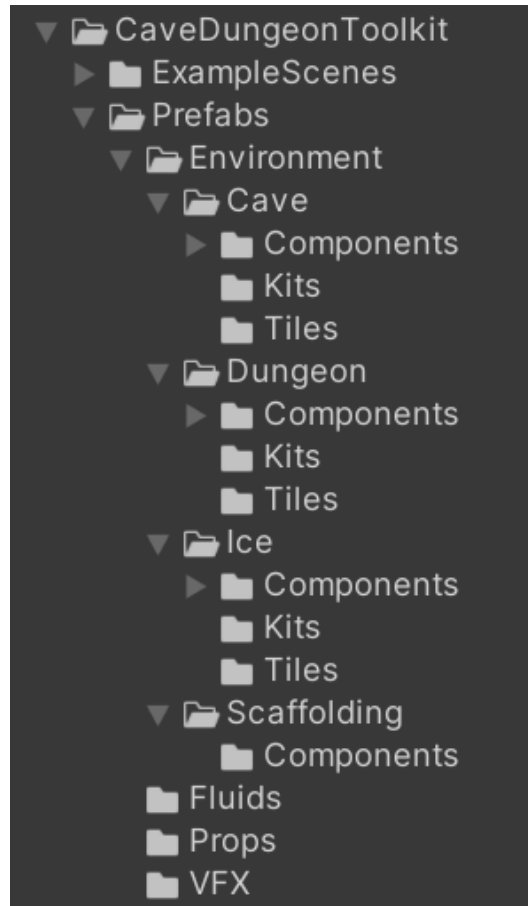
If lighting looks weird, the pixel light count might be too low. URP and Built in have different processes for this.

- **Built-in:** Under Project Settings > Quality, crank up the pixel light count.
- **URP:** Select your render pipeline asset, under "Lighting" make sure "Additional Lights" is set to "Per Pixel" and crank up the per object limit.

Note this is pretty unoptimized, check out the performance optimization section of the documentation for more information about this.

In the LostTemple scene, the terrain will be pink. Select the terrain ("Terrain" in the hierarchy) and under "Material" in the terrain settings, select "Create" and select one of the terrain shaders. If it's the wrong shader, Unity will give a warning about how to fix this. Follow what that says.

Using the assets



There are more folders included in this package, but for creating scenes all you need to worry about are these folders. Here's a breakdown on how to best use these:

- **ExampleScenes:** This is a good place to start. Digging through these scenes shows some different ways you can use these assets to create levels.
- **Kits:** These are a good way to start building scenes. Kits are collections of components that can speed up basic level creation.
- **Tiles:** An easier alternative to kits that snap nicely to 5x or 10x grids. Should be usable in grid based tools or dungeon generators.
- **Components:** These are individual assets that can assist in fleshing out your scene and adding detail. MicroComponents are usually smaller parts of components for adding fine detail.
- **Props:** These are pieces of furniture and objects that can help flesh out a scene.
- **Fluids:** Water and lava objects.
- **VFX:** Most of these are atmospheric effects that can be placed to add depth and movement to the scene.

Cave and Ice Prefabs

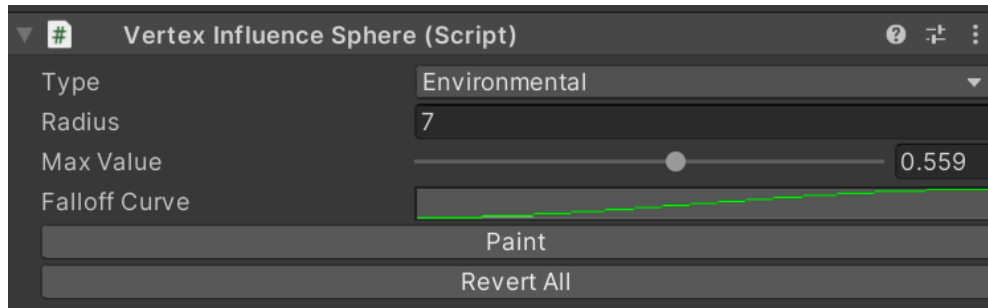
Most of the other prefabs are pretty self explanatory to use, but you might be a bit confused about how to use the “cave” and “ice” component prefabs well. This goes over their general usage.

- **Boulders:** These are good to place almost anywhere to help break up repetition and create interesting shapes.
- **Floor Breakup:** As the name implies, these can be places all over the ground to create some nice variation and break repetition.
- **Floor Rock Pile:** These are world aligned pieces that can help break up the floor and add more detail. Try replacing the material on these with the bigger or smaller variation for a different look, or with the ice variations.
- **Freeform:** These are very versatile and useful assets that can be scaled up almost infinitely to help create very large scenes. They can be placed alongside wall or ceiling assets to break repetition or just be used by themselves to create holes or dynamic shapes.
- **Platforms:** These assets can be combined to create organic medium to large sized platforms and outcroppings
- **Small Rocks:** Similar to boulders, but on a smaller scale.
- **Squares:** Blockier rocks useful for unique variation. Square1 in particular is pretty versatile and can be used to create jagged cliffs or nice corners.
- **Stacked Blocks:** These can be used for outcroppings, breaking up floor pieces or stacked to create stairs or platforms.
- **Stalactites:** What cave is complete without them? You can get individual stalactites that aren't grouped together in the MicroComponents folder. (Spoiler warning: these can also be used as stalagmites. Don't tell anyone though!)

Using the Tools

Vertex Influence Spheres

These allow you to paint environmental effects. Add them to your scene by selecting GameObject > VertexInfluenceSphere (or by adding VertexInfluenceSphere component to a gameobject).



Type: This can be Environmental or Wetness. Environmental will blend in an environmental effect specified within the CaveDungeonFunctions window. Wetness will add glossiness that will make objects look more wet.

Radius: The radius, in Unity units (meters).

MaxValue: The maximum value outputted. The effect will be more drastic with a higher value and more subtle with a lower value

Falloff Curve: This can be used to get more control out of the vertex influence sphere. The curve represents the falloff of the sphere painting, with the left side being the extents of the sphere radius and the right side being the center point of the sphere. This is multiplied by the MaxValue set above.

Paint: Paints all the vertex spheres in the scene. Press this to update any changes made.

Revert All: Removes the vertex colors added by the influence spheres from the scene. Simply paint again to bring them back. You should do this before deleting vertex influence spheres from the scene.

Material Blend Setter

To change what type of material is blended in the scene, you need a Material Blend Setter. To get this, create an empty GameObject and add the Material Blend Type Setter script to it.

With this, you can choose between blending snow, sand or moss in the scene. Additionally, you can select "Is Desert" to give all rocks and bricks a reddish sandstone tint. These changes should instantly apply after changing them in this script.

Advanced Vertex Influence Sphere Info:

Vertex Influence spheres paint to the red channel of the vertex colors of the mesh for environmental blending and the green channel of the vertex colors for the wetness effect.

This system creates and maintains a copy of the mesh whenever vertex colors are changed either due to material swapping or vertex painting. To determine which objects can be painted and to store the original mesh to revert, a script is attached to each object called “MeshStorage”.

If you want to have additional meshes that use this system, then your mesh should by default have black painted in the vertex colors and have the MeshStorage script attached to it.

The Material Blend Setter changes global keywords in the scene that the shaders use to statically switch certain aspects of the shader. They are also multi-compile keywords so this behavior works in builds. You can see how this all works under the hood by opening MaterialBlendTypeSetter.cs and any of the shaders that use this in the ShaderGraph, like Master.

I set it up this way to make the example scenes work the easiest. However, your project might have different needs. If you know about shader keywords, it shouldn't be too hard to manually change yourself by editing the shadergraphs. More information about keywords:

<https://docs.unity3d.com/Packages/com.unity.shadergraph@12.0/manual/Keywords.html>

Performance Optimization

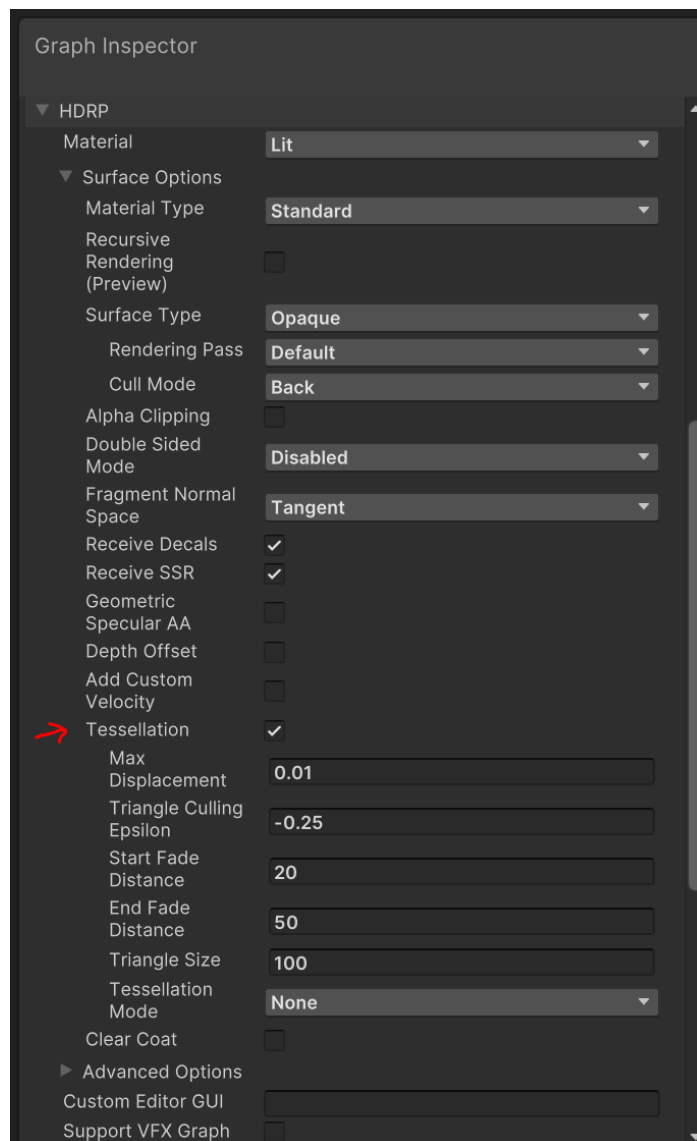
For this package, I wanted to make available the highest spec anyone would need for these assets. However, in some cases (especially in a VR environment) some optimization might be needed.

To start with, you should make sure you're taking advantage of how Unity optimizes draw calls. This will be different depending on the render pipeline, but these assets by default are marked

as static and should be able to take advantage of most of these. Just note that vertex painted meshes are unique meshes, so mesh instancing will be less effective in these cases. For more information, read <https://docs.unity3d.com/Manual/optimizing-draw-calls.html>.

One of the more expensive effects in the HDRP version is the tessellation shader, which can increase polygon count by a lot if polygon count is a performance issue. To start with, you can go through the “materials” folder and change any material with “CDT/MasterTessellation” with “CDT/Master” (or MasterTessellationIc with MaterIc). This covers most objects with tessellation, but some other shaders also use this. Under “shaders”, find these shaders and uncheck tessellation in the shader graph.

- Freeform
- FreeformIc
- FloorRocks
- VFX_LavaLake



Another aspect that may be performance intensive is the size of some of the textures. Again, I wanted the textures to be as big as anyone would want them, but in a lot of cases like in third person or top down games, you might not need this high resolution. You can change the resolution of the textures using Unity's "Max Size" option for textures. A good starting point might be to select all the textures and set the max size to 1024, or 512. From there, if you find some assets to be too blurry, you can up the texture resolution for those specific assets.

Especially in HDRP, I've noticed transparent shaders cause a considerable performance drop. My scenes usually have a "VFX" group that holds all the transparent effects. See if deleting this gains any performance and is worth the tradeoff.

Finally, my example scenes use a lot of lights, which may cause poor performance depending on your render pipeline and project configuration. Consider using baked lighting if you notice a significant drop in performance relating to turning the lights on or off.

Build Optimization

Before you start a build, I **highly** recommend you remove unused render pipeline targets from the shaders. These add a **lot** of time to the build time and are only included so this package has compatibility across all render pipelines.

Open each shader in ShaderGraph. In the Graph Inspector, there will be 3 Active Targets. Delete all of them except the one you're using (select it and hit the "-" button).

